15 mark questions and answers

1. BRESENHAM'S LINE DRAWING ALGORITHM

Bresenham's Line Drawing Algorithm is one of the most important algorithms in computer graphics used to draw a straight line on raster displays. It was developed by **Jack E. Bresenham in 1965**.

In raster graphics, a line must be represented using pixels. This algorithm selects the **closest pixel to the actual line path** using only **integer calculations**, making it faster and more efficient than the DDA algorithm.

Basic Idea

At each step, the algorithm decides which pixel is closer to the true line:

- **E (East)** → (x+1, y)
- **NE (North-East)** → (x+1, y+1)

This decision is made using a **decision parameter (p)**.

Assumptions

- Line from (x_1, y_1) to (x_2, y_2)
- Slope $0 < m < 1$
- $dx = x_2 - x_1, dy = y_2 - y_1$

Mathematical Expression

Decision parameter:

$$p_0 = 2dy - dx$$

Algorithm

START

1. Input endpoints (x_1, y_1) and (x_2, y_2)
2. Calculate:
    - $dx = x_2 - x_1$
    - $dy = y_2 - y_1$
3. Initialize decision parameter:
    - $p = 2dy - dx$
4. Set initial point:
    - $x = x_1, y = y_1$
5. Plot (x, y)

6. WHILE ($x < x_2$) DO
   a. IF ($p < 0$) THEN
      - $x = x + 1$
      - $p = p + 2dy$
      b. ELSE
      - $x = x + 1$
      - $y = y + 1$
      - $p = p + 2dy - 2dx$
      c. Plot (x, y)
7. END WHILE

STOP

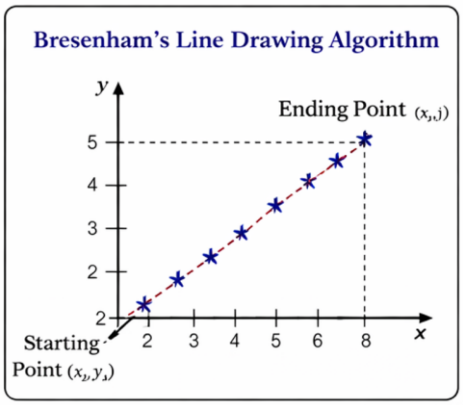
Worked Example

Draw a line from (2,2) to (8,5)

- $dx = 6$, $dy = 3$
- $p_0 = 2 \times 3 - 6 = 0$

Step	x	y	p	Decision
0	2	2	0	Start
1	3	3	-6	NE
2	4	3	0	E
3	5	4	-6	NE
4	6	4	0	E
5	7	5	-6	NE
6	8	5	—	End

Diagram



Advantages

1. Uses only integer arithmetic
2. Faster than DDA
3. No floating-point calculations
4. Produces accurate lines
5. Efficient for real-time graphics

Disadvantages

1. Works directly only for slope < 1
2. Needs modification for other slopes

Applications

- Computer graphics systems
- CAD (Computer-Aided Design)
- Game development
- Drawing software

2. RASTER SCAN SYSTEM vs RANDOM SCAN SYSTEM (FULL 15 MARK)

Display systems are essential components of computer graphics used to present images on the screen. There are two major types of display systems:

1. **Raster Scan System**
2. **Random Scan System**

These systems differ in how the electron beam scans and displays images.

RASTER SCAN SYSTEM

Raster scan system is a display method in which the screen is divided into a grid of pixels, and the image is displayed by scanning the screen **line by line from top to bottom**.

Working Principle

- The image is stored in a **frame buffer (refresh buffer)**
- The electron beam scans horizontally across the screen
- After completing one line, it moves to the next line
- This continues until the entire screen is scanned
- The process repeats continuously (refreshing)

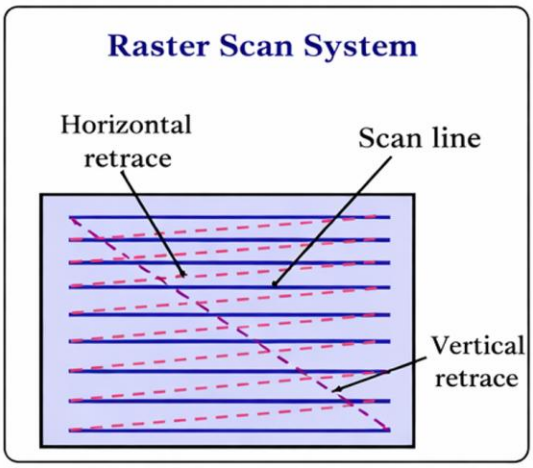
Algorithm (START–STOP Format)

START

1. Store image data in frame buffer
2. Position beam at top-left corner
3. Scan horizontally (left → right)
4. Turn beam ON/OFF based on pixel value
5. Move to next scan line
6. Repeat steps 3–5 until last line
7. Refresh continuously

STOP

Diagram



Advantages

1. Suitable for realistic images
2. Supports color, shading, and filled areas
3. Easy to implement
4. Widely used in TVs and monitors

Disadvantages

1. Requires large memory (frame buffer)
2. Lower resolution compared to vector display
3. May produce jagged edges (aliasing)

RANDOM SCAN SYSTEM

Random scan system (also called **vector display**) draws images by directing the beam **only along the lines to be drawn**, instead of scanning the entire screen.

Working Principle

- The beam moves directly to the required position

- Draws lines between specified points
- No need for pixel-by-pixel scanning
- Uses display file instead of frame buffer

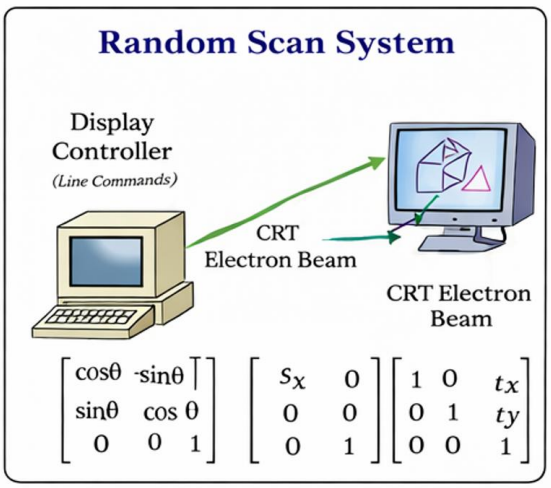
Algorithm

START

1. Store line drawing commands
2. Move beam to starting point
3. Turn beam ON
4. Draw line to endpoint
5. Turn beam OFF
6. Repeat for all lines
7. Refresh display

STOP

Diagram



Advantages

1. Produces smooth and sharp lines
2. High resolution
3. Requires less memory
4. No aliasing effect

Disadvantages

1. Cannot display filled areas
2. Not suitable for complex images

3. Expensive hardware
4. Limited color capability

COMPARISON TABLE (VERY IMPORTANT)

Feature	Raster Scan	Random Scan
Drawing Method	Pixel-based	Line-based
Memory	High (frame buffer)	Low
Resolution	Moderate	High
Image Type	Filled images	Line drawings
Speed	Slower	Faster
Cost	Low	High

3. 2D TRANSFORMATIONS USING HOMOGENEOUS COORDINATES

In computer graphics, **2D transformations** are used to change the **position, size, orientation, and shape** of objects in a 2D plane.

These transformations are performed using **matrix operations**. To make calculations easier and to combine multiple transformations efficiently, we use **homogeneous coordinates**.

Basic Idea

A point (x, y) is represented in homogeneous form as:

$$(x, y) \rightarrow (x, y, 1)$$

This allows all transformations (translation, rotation, scaling) to be represented using **3×3 matrices**.

Types of 2D Transformations

1. Translation

Moves an object from one position to another.

Transformation Equation

$$x' = x + t_x, y' = y + t_y$$

Matrix Form

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

2. Scaling

Changes the size of an object.

Equation

$$x' = x \cdot s_x, y' = y \cdot s_y$$

Matrix Form

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3. Rotation

Definition

Rotates an object about the origin.

Equation

$$x' = x\cos\ \theta - y\sin\ \theta$$
$$y' = x\sin\ \theta + y\cos\ \theta$$

Matrix Form

$$R = \begin{bmatrix} \cos\theta & -\sin\ \theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Algorithm

START

1. Represent all points in homogeneous form (x, y, 1)
2. Select the required transformation (translation/scaling/rotation)
3. Construct the transformation matrix
4. Multiply matrix with each point
5. Obtain new transformed coordinates
6. Repeat for all vertices of object

STOP

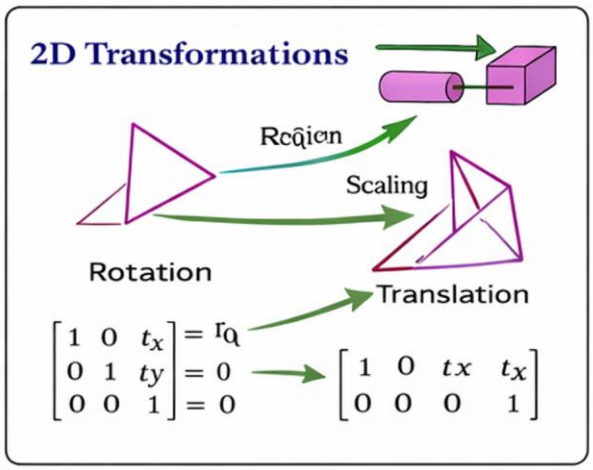
Composite Transformation

Multiple transformations can be combined into a **single matrix**:

$$M = T \times R \times S$$

This reduces computation and improves efficiency.

Diagram



Advantages of Homogeneous Coordinates

1. All transformations use **matrix multiplication**
2. Easy to combine transformations
3. Efficient computation
4. Suitable for computer implementation

Applications

- Animation
- CAD systems
- Image processing
- Game graphics

Disadvantages

1. Extra coordinate increases computation slightly
2. Needs understanding of matrix operations

4. 2D VIEWING PIPELINE & WINDOW TO VIEWPORT TRANSFORMATION

In computer graphics, objects are created in **world coordinates**, but they must be displayed on a screen. The process of converting these coordinates into screen coordinates is called the **2D Viewing Pipeline**.

A key step in this process is the **Window to Viewport Transformation**, which maps a selected part of the scene onto the display device.

2D VIEWING PIPELINE

The 2D viewing pipeline is a sequence of steps used to convert object coordinates into screen coordinates for display.

Steps in Viewing Pipeline

1. Object Definition

- Objects are defined using vertices and shapes
- Created in **model coordinates**

2. World Coordinate Transformation

- Objects are placed in the scene
- Converted into **world coordinates**

3. Window Selection (Viewing Window)

- Select a portion of the world to display
- Defined by:
    - x_{wmin}, x_{wmax}
    - y_{wmin}, y_{wmax}

4. Clipping

- Remove parts outside the window
- Improves efficiency

5. Viewport Transformation

- Map window to screen (viewport)
- Converts coordinates into display space

Algorithm (Viewing Pipeline)

START

1. Define objects in model coordinates
2. Convert to world coordinates
3. Select viewing window
4. Perform clipping
5. Define viewport on screen
6. Apply window-to-viewport transformation
7. Display final image

STOP

WINDOW TO VIEWPORT TRANSFORMATION

It is the process of mapping a selected window in world coordinates to a viewport in device coordinates.

Transformation Formula (VERY IMPORTANT)

$$x_v = x_{vmin} + (x_w - x_{wmin}) \cdot \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}}$$
$$y_v = y_{vmin} + (y_w - y_{wmin}) \cdot \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}$$

- (x_w, y_w)→ window coordinates
- (x_v, y_v)→ viewport coordinates
- Ensures **proper scaling and positioning**

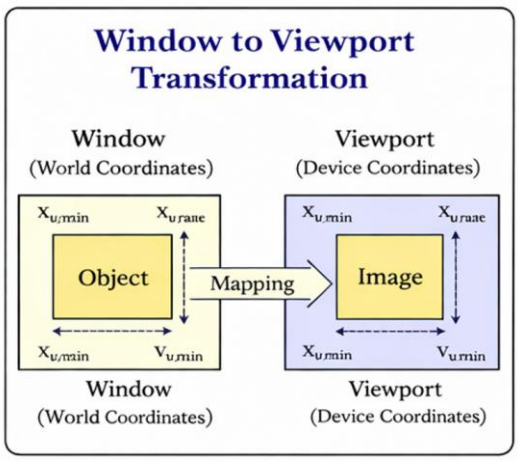
Algorithm (Window → Viewport)

START

1. Input window limits: $x_{wmin}, x_{wmax}, y_{wmin}, y_{wmax}$
2. Input viewport limits: $x_{vmin}, x_{vmax}, y_{vmin}, y_{vmax}$
3. For each point (xw, yw):
   a. Compute xv using formula
   b. Compute yv using formula
4. Plot (xv, yv) on screen

STOP

Diagram (VERY IMPORTANT — MUST DRAW)



Advantages

1. Efficient display of selected area
2. Reduces unnecessary computation

3. Maintains aspect ratio
4. Enables zooming and panning

Applications

- Computer graphics systems
- Game development
- CAD software
- Image processing

Disadvantages

1. Slight computation overhead
2. Requires proper clipping

5. 3D OBJECT REPRESENTATION METHODS

In computer graphics, 3D objects must be represented in a form that a computer can store, process, and display. This is called **3D object representation**.

Different methods are used depending on accuracy, memory, and application requirements.

Need for 3D Representation

- To model real-world objects
- To perform rendering and visualization
- To support CAD, animation, and simulation

Types of 3D Object Representation

1. Polygon Surface Representation

Objects are represented using **vertices, edges, and polygonal faces**.

- Surface is divided into polygons (usually triangles or rectangles)
- Each polygon is defined by its vertices

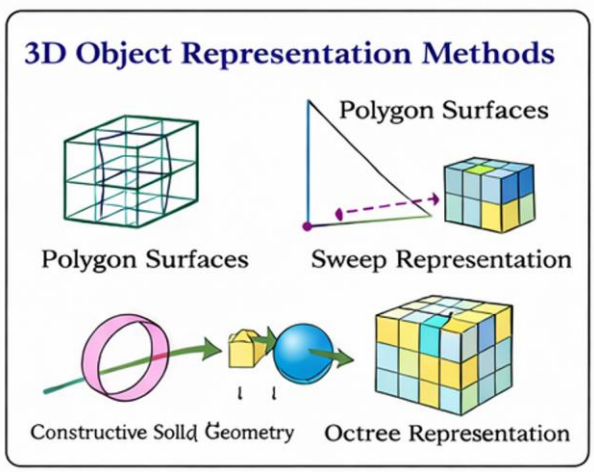
Algorithm

START

1. Define vertices of object
2. Connect vertices to form edges
3. Combine edges to form polygon faces
4. Store polygons in data structure
5. Display object

STOP

Diagram



Advantages

- Simple and widely used
- Easy to implement
- Efficient for rendering

Disadvantages

- Cannot represent curved surfaces accurately
- Needs many polygons for smooth objects

2. Sweep Representation

A 3D object is created by **moving a 2D shape along a path**.

Types

- Translational sweep
- Rotational sweep

Algorithm (START–STOP)

START

1. Define 2D shape
2. Define path (line or curve)
3. Move shape along path
4. Generate surface
5. Form 3D object

STOP

Advantages

- Easy to create complex objects
- Useful for engineering models

Disadvantages

- Limited to specific shapes
- Complex paths increase computation

3. Constructive Solid Geometry (CSG)

Objects are created using **Boolean operations** on simple shapes.

Basic Operations

- Union
- Intersection
- Difference

Algorithm (START–STOP)

START

1. Define basic primitives (cube, sphere, cylinder)
2. Select Boolean operation
3. Apply operation between objects
4. Generate final object
5. Store result

STOP

Advantages

- Produces precise models
- Easy modification
- Suitable for CAD

Disadvantages

- Complex calculations
- Hard to visualize intermediate steps

4. Octree Representation

Space is recursively divided into **8 smaller cubes (octants)**.

- Used for spatial partitioning
- Efficient for visibility and rendering

Algorithm

START

1. Consider entire space as cube
2. Divide into 8 sub-cubes
3. Check each cube: inside/outside object
4. Subdivide further if needed
5. Store structure

STOP

Advantages

- Efficient memory usage
- Faster processing

Disadvantages

- Complex structure
- Difficult implementation

Comparison (Write if time permits)

Method	Feature
Polygon	Simple, widely used
Sweep	Path-based modeling
CSG	Boolean operations
Octree	Space partitioning

Applications

- CAD systems
- Animation
- Gaming
- Simulation

6. FLAT PANEL DISPLAYS (FULL 15 MARK ANSWER)

Flat Panel Displays (FPDs) are modern display devices that are **thin, lightweight, and compact** compared to traditional CRT displays.

They are widely used in:

- Computers
- Laptops
- Mobile phones
- Televisions

Flat panel displays are designed to reduce space, power consumption, and radiation.

Types of Flat Panel Displays

Flat panel displays are mainly classified into:

1. **Emissive Displays**
2. **Non-Emissive Displays**

DIAGRAM

```
Light Source (Backlight)
        ↓
  -----------------------
  |   Polarizer     |
  -----------------------
        ↓
  -----------------------
  | Liquid Crystal Layer|
  -----------------------
        ↓
  -----------------------
  |  Color Filter   |
  -----------------------
        ↓
  -----------------------
  |     Screen     |
  -----------------------
```

1. EMISSIVE DISPLAYS

Definition

Emissive displays are devices that **produce light by themselves** to display images.

Examples

- Plasma Panel
- LED displays

Working Principle

- Electrical energy excites gas or material
- Light is emitted directly

- Pixels glow to form images

Algorithm

START

1. Apply electrical signal to pixels
2. Excite gas/material inside panel
3. Emit light from pixels
4. Control brightness and color
5. Form image on screen

STOP

Diagram

Electrical Signal → Pixel → Light Emission → Image

Advantages

1. High brightness
2. Wide viewing angle
3. Good color quality

Disadvantages

1. High power consumption
2. Heat generation
3. Expensive

2. NON-EMISSIVE DISPLAYS

Definition

Non-emissive displays do **not produce light themselves**, but use external light and control it to display images.

Example

- LCD (Liquid Crystal Display)

Working Principle

- Light source (backlight) is used
- Liquid crystals control light passage
- Image is formed by blocking/allowing light

Algorithm

START

1. Turn on backlight
2. Apply voltage to liquid crystals
3. Crystals align accordingly
4. Control light passing through
5. Form image on screen

STOP

Diagram (VERY IMPORTANT)

Backlight → Liquid Crystal → Filter → Screen

Advantages

1. Low power consumption
2. Thin and lightweight
3. No radiation
4. Portable

Disadvantages

1. Limited viewing angle
2. Lower brightness than emissive
3. Response time issues

Comparison Table

Feature	Emissive	Non-Emissive
Light Source	Self	External
Power	High	Low
Brightness	High	Moderate
Example	Plasma	LCD

Applications

- Mobile phones
- Laptops
- TVs
- Digital displays